

MAST Guide

Android Application Security Testing Guide

Phase 1: Information Gathering & Reconnaissance

Objective: This initial phase is about understanding the application's landscape without running it in a dynamic environment. We aim to decompile the application package (APK), map its components (Activities, Services, etc.), identify its permissions, and discover its attack surface. A thorough reconnaissance phase is critical as it informs our entire testing strategy.

Activity 1.1: Analyze `AndroidManifest.xml`

Main Heading: `MASVS-PLATFORM` - Platform Interaction

Sub-heading: `MASTG-PLAT-01` - Review `AndroidManifest.xml` for Security Misconfigurations

Objective: To inspect the `AndroidManifest.xml` file. This file is the "blueprint" of the app, declaring its components, permissions, and fundamental security configurations.

Impact:

- **High Risk:** A misconfiguration here can expose the entire application. For example, setting the app as `debuggable` allows an attacker to attach a debugger and tamper with its runtime memory.
- **Medium Risk:** Improperly exported components can be accessed by malicious apps, and allowing backups can lead to sensitive data extraction.

Tooling:

- `jadx-gui`
- `MobSF` (Mobile Security Framework)
- `apktool`

Methodology:

1. Obtain the `.apk` file of the target application.
2. Open the `.apk` file using `jadx-gui`.
3. In the navigation pane, find and click on `resources/AndroidManifest.xml` to view its contents.
4. **Check for Debugging:** Look for the `android:debuggable` attribute within the `<application>` tag. In a production build, this must be set to `false`.
5. **Check for Backups:** Look for the `android:allowBackup` attribute. If the app handles sensitive data, this should be `false` to prevent users from extracting app data via `adb backup`.

6. **Check for Cleartext Traffic:** Look for `android:usesCleartextTraffic`. If set to `true`, the app is allowed to send network traffic over unencrypted channels (like HTTP).
7. **Enumerate Components:** Review all `<activity>`, `<service>`, `<receiver>`, and `<provider>` tags. Look for any component that includes the `android:exported="true"` flag, as this means it can be invoked by other apps on the device.

Unsuccessful Response (Vulnerable Configuration):

The `AndroidManifest.xml` contains multiple insecure flags.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ...>
  <application
    android:allowBackup="true"
    android:debuggable="true"
    android:usesCleartextTraffic="true" ...>

    <activity
      android:name=".SecretDebugActivity"
      android:exported="true" />
    <provider
      android:name=".InsecureFileProvider"
      android:exported="true"
      android:grantUriPermissions="true" />

  </application>
</manifest>
```

Successful Response (Secure Configuration):

The manifest is hardened, explicitly disabling debug mode and backups, and not exporting any components unnecessarily.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ...>
  <application
    android:allowBackup="false"
    android:debuggable="false"
    android:usesCleartextTraffic="false" ...>

    <activity
      android:name=".SecretDebugActivity"
      android:exported="false" />
    <provider
      android:name=".SecureFileProvider"
      android:exported="false" />

  </application>
</manifest>
```

```
</application>
</manifest>
```

Activity 1.2: Decompile APK & Review Source

Main Heading: MASVS-CODE - Code Quality

Sub-heading: MASTG-CODE-01 - Decompile APK for Static Analysis

Objective: To decompile the APK's Dalvik Executable (`.dex`) files into human-readable Java code. This allows for a static (white-box) review of the application's logic, which is the foundation for finding most vulnerabilities.

Impact:

- **Critical Risk (Reconnaissance Finding):** Successful decompilation is the gateway to finding critical flaws like hardcoded API keys, insecure cryptographic implementations, and business logic vulnerabilities.

Tooling:

- `jadx` (command-line) or `jadx-gui` (GUI)
- `Bytecode Viewer` (an alternative that bundles multiple decompilers)

Methodology:

1. Open your terminal.
2. Use the `jadx-gui` command to open the APK in the graphical interface, which is excellent for browsing code.

```
jadx-gui /path/to/target-app.apk
```

3. Alternatively, use the `jadx` command-line tool to decompile the entire application into a directory for review in a code editor like VS Code.

```
jadx -d /path/to/output-directory /path/to/target-app.apk
```

4. Once decompiled, navigate the source code. Pay special attention to files related to `network`, `storage`, `crypto`, and `auth`.

Successful Response (Tester's "Win"):

The decompiler successfully processes the APK and outputs readable Java source code.

```
$ jadx -d ./app-source ./target-app.apk
INFO - loading ...
INFO - processing ...
INFO - decompiling ...
```

```
INFO - saving ...  
INFO - done
```

You can now browse the full Java source in the `./app-source` directory.

Unsuccessful Response (Tester's "Loss"):

The application employs strong anti-reverse-engineering controls (a topic for Phase 4). The decompiler fails or produces unreadable code (known as Smali).

```
$ jadx -d ./app-source ./target-app.apk  
INFO - loading ...  
WARN - class 'a.a.b.a' processing error: ...  
ERROR - finished with errors, count: 5
```

(Note: This is a "Successful Response" from a defensive standpoint. Our goal as testers is to bypass this.)

Activity 1.3: Identify App Components & Attack Surface

Main Heading: `MASVS-PLATFORM` - Platform Interaction

Sub-heading: `MASTG-PLAT-02` - Enumerate Application Attack Surface with `drozer`

Objective: To identify all exported application components (Activities, Services, Broadcast Receivers, Content Providers) that act as entry points and can be invoked by other apps on the device.

Impact:

- **High Risk:** A single insecure, exported component can lead to data leakage, unauthorized actions, or a full application compromise. This is a primary vector for privilege escalation.

Tooling:

- `drozer`
- `adb` (Android Debug Bridge)

Methodology:

1. Install the `drozer` agent `.apk` on your test device/emulator.
2. On your workstation, connect to the device and forward the `drozer` port:

```
adb forward tcp:31415 tcp:31415
```

3. Launch the `drozer` agent on the device and start a console session:

```
drozer console connect
```

4. Find the package name of your target app (e.g., `com.example.vulnerable`).

5. Run the `attacksurface` module to get a summary of exposed components:

```
run app.package.attacksurface com.example.vulnerable
```

6. For each exported component type, get more detailed information:

```
run app.activity.info -a com.example.vulnerable
run app.service.info -a com.example.vulnerable
run app.broadcast.info -a com.example.vulnerable
run app.provider.info -a com.example.vulnerable
```

Unsuccessful Response (Vulnerable Application):

The `drozer` output reveals a large and dangerous attack surface.

```
dz> run app.package.attacksurface com.example.vulnerable
Attack Surface of com.example.vulnerable:
  3 activities exported
  1 services exported
  2 broadcast receivers exported
  1 content providers exported
  is debuggable [!!!]
```

Successful Response (Secure Application):

The application follows the principle of least privilege, exporting no components.

```
dz> run app.package.attacksurface com.example.secure
Attack Surface of com.example.secure:
  0 activities exported
  0 services exported
  0 broadcast receivers exported
  0 content providers exported
  is not debuggable
```

Activity 1.4: Map Network Communication & Endpoints

Main Heading: `MASVS-NETWORK` - Network Communication

Sub-heading: `MASTG-NET-01` - Identify and Map API Endpoints

Objective: To identify all API endpoints, URLs, and domains the application communicates with. This is a reconnaissance step to prepare for Phase 3 (Dynamic Analysis).

Impact:

- **Medium Risk (Information Leakage):** Hardcoding development, staging, or internal-only API endpoints into a production app is a common and dangerous information leak. It gives an attacker a map of your internal infrastructure.

Tooling:

- **MobSF** (Mobile Security Framework)
- **jadx** (for manual code review)
- **Burp Suite** (for dynamic mapping)

Methodology:

This is a static reconnaissance activity.

1. **Automated Scan:** Upload the `.apk` file to a **MobSF** instance. After the scan, navigate to the "Reconnaissance" -> "URLs" section to see a list of all URLs MobSF automatically extracted from the code and other files.
2. **Manual Code Review:** Use the decompiled code (from Activity 1.2) to manually search for strings that indicate network communication.
3. In your code editor, search the entire codebase for keywords like:
 - `https://`
 - `http://`
 - `api.`
 - `example.com` (replace with the target's domain)
 - `Socket(`
4. Compile a master list of all unique domains and API endpoints discovered.

Unsuccessful Response (Vulnerable Information Leak):

The automated scan or manual review reveals hardcoded HTTP URLs and endpoints for non-production environments.

MobSF Output:

```
[INFO] URLs
https://api.production.com/v2/login
https://api.production.com/v2/users
http://dev-api.internal.com/v1/test
https://analytics.third-party.com/
[HIGH] Hardcoded HTTP URL(s) found.
      - http://dev-api.internal.com/v1/test
```

Successful Response (Secure Application):

The scan reveals no hardcoded URLs, or only reveals the single, expected production API endpoint.

MobSF Output:

```
[INFO] URLs
https://api.production.com/v2/
[INFO] No Hardcoded HTTP URLs found.
```

This completes the initial reconnaissance phase. I am now ready to move on to **Phase 2: Static Analysis**, where we will use this information to find specific vulnerabilities in the decompiled code, focusing on `MASVS-STORAGE` (insecure data storage) and `MASVS-CODE` (hardcoded secrets).

Phase 2: Static Analysis (Code Review)

Objective: To analyze the application's decompiled source code (from Phase 1) without executing it. This phase is about finding vulnerabilities that are "at rest" within the code, such as hardcoded secrets, insecure data storage practices, and weak cryptographic implementations.

Activity 2.1: Test for Insecure Data Storage in `SharedPreferences`

Main Heading: `MASVS-STORAGE-1` - The app securely stores sensitive data.

Sub-heading: `MASTG-TEST-0287` - Sensitive Data Stored Unencrypted via the `SharedPreferences` API.

Objective: To inspect the decompiled code for any sensitive data (credentials, session tokens, PII, API keys) being written in plaintext to `SharedPreferences` files.

Impact:

- **High Risk:** `SharedPreferences` are stored as plaintext XML files in the app's sandboxed data directory. On a rooted device, any application or an attacker with `adb` access can read these files and steal the credentials.

Tooling:

- `jadx-gui` (or any code editor with the decompiled source).

Methodology:

1. Using the decompiled source from Phase 1, perform a global search for the string `getSharedPreferences`.
2. Follow the references to where the app *writes* data, looking for methods like `putString()`, `putInt()`, etc.
3. Analyze the data being written. Look for variable names like `token`, `password`, `session`, `apiKey`, or `user_id`.
4. If sensitive data is being written directly, the application is vulnerable.

Unsuccessful Response (Vulnerable Application):

The code review reveals a `login` function that stores the user's password and session token in plaintext.

```
// From /com/example/vulnerable/SessionManager.java
```

```

public void createLoginSession(String username, String password, String token) {
    Editor editor = this.prefs.edit();
    editor.putString("username", username);
    editor.putString("password", password); // [!!!] CRITICAL: Storing password in
plaintext
    editor.putString("session_token", token); // [!!!] HIGH: Storing token in
plaintext
    editor.commit();
}

```

Successful Response (Secure Application):

The application does not store sensitive data in `SharedPreferences`. Alternatively, if it must store a token, it uses an encrypted `SharedPreferences` implementation.

```

// From /com/example/secure/SessionManager.java

// Using androidx.security.crypto.EncryptedSharedPreferences
SharedPreferences encryptedPrefs = EncryptedSharedPreferences.create(
    "secure_prefs",
    masterKeyAlias,
    context,
    EncryptedSharedPreferences.PrefKeyEncryptionScheme.AES256_SIV,
    EncryptedSharedPreferences.PrefValueEncryptionScheme.AES256_GCM
);

public void createLoginSession(String token) {
    Editor editor = encryptedPrefs.edit();
    // The data is automatically encrypted before being written to disk
    editor.putString("session_token", token);
    editor.commit();
}

```

Activity 2.2: Test for Hardcoded Secrets

Main Heading: `MASVS-CODE` - Code Quality & Build Settings

Sub-heading: `MASTG-CODE-02` - Test for Hardcoded Secrets

Objective: To identify any secrets (API keys, encryption keys, credentials, tokens) that are hardcoded directly into the application's source code or resource files.

Impact:

- **Critical Risk:** Hardcoded secrets are a critical vulnerability. An attacker can easily decompile the app (as shown in Phase 1) and extract these keys. This can lead to a full compromise of the

application's backend API, third-party services, or encrypted data.

Tooling:

- `jadx-gui`
- `MobSF`
- `grep` (command-line search)

Methodology:

1. **Automated Scan:** Upload the `.apk` file to `MobSF`. After the scan, `MobSF` will provide a "Hardcoded Secrets" section listing high-entropy strings it found.
2. **Manual Code Review:** In the decompiled `jadx-gui` source, search for common keywords:
 - `apiKey`
 - `api_key`
 - `secret`
 - `password`
 - `bearer`
 - `client_id`
 - `client_secret`
3. **Check Resource Files:** In `jadx-gui`, navigate to `resources/res/values/strings.xml`. This is a common place developers mistakenly store keys to "avoid" hardcoding them in Java, but it is just as insecure.

Unsuccessful Response (Vulnerable Application):

A manual review of the code and resource files reveals multiple hardcoded secrets.

```
// From /com/example/vulnerable/ApiClient.java
```

```
public class ApiClient {  
    // [!!!] CRITICAL: Hardcoded API key  
    private final String API_KEY = "ak_live_123abc456xyz789def";  
  
    // [!!!] CRITICAL: Hardcoded encryption key  
    private final byte[] AES_KEY = new byte[]{ 1, 2, 3, 4, 5, 6, 7, 8, ... };  
}
```

```
<resources>  
    <string name="app_name">Vulnerable App</string>  
    <string name="google_maps_key">AIzaSyB...[redacted]</string>  
</resources>
```

Successful Response (Secure Application):

No secrets are found in the code or resource files. A secure app fetches keys from a secure build configuration (like `build.gradle`) or a server-side endpoint.

```
// From /com/example/secure/ApiClient.java

public class ApiClient {
    // Key is loaded from BuildConfig, which is populated at build time
    // and not part of the committed source code.
    private final String API_KEY = BuildConfig.API_KEY;
}

<resources>
    <string name="app_name">Secure App</string>
    <string name="login_prompt">Please log in</string>
</resources>
```

Activity 2.3: Test for Sensitive Data Leakage via Logging

Main Heading: `MASVS-STORAGE-2` - The app prevents leakage of sensitive data.

Sub-heading: `MASTG-TEST-0231` - References to Logging APIs.

Objective: To find instances where the application logs sensitive data (credentials, tokens, PII, network responses) to the system log (`logcat`).

Impact:

- **High Risk:** In production builds, logging should be removed or sanitized. Any app with the `android.permission.READ_LOGS` (on older Android versions) or an attacker with `adb` access can read the system-wide `logcat` buffer and steal this data in real-time.

Tooling:

- `jadx-gui`

Methodology:

1. In the decompiled `jadx-gui` source, perform a global search for all instances of the Android logging class: `Log`.
2. Review every occurrence of `Log.d`, `Log.v`, `Log.i`, `Log.w`, and `Log.e`.
3. Pay close attention to what is being logged. Look for concatenation of strings with variables like `password`, `token`, `responseBody`, or `cookie`.
4. A secure application will have logging removed for production builds, often by using a tool like **ProGuard**. Check for a `proguard-rules.pro` file that contains rules to strip `Log` calls.

Unsuccessful Response (Vulnerable Application):

The code review reveals that sensitive user credentials and network data are being logged.

```
// From /com/example/vulnerable/LoginActivity.java

public void onLoginClick(String username, String password) {
    // [!!!!] CRITICAL: Logging username and password in plaintext
    Log.d("VULN_APP", "Attempting login for user: " + username + " with pass: " +
password);

    // ...
}

// From /com/example/vulnerable/ApiClient.java
public void onResponse(Call call, Response response) {
    String responseData = response.body().string();

    // [!!!!] HIGH: Logging the raw server response, which may contain session
tokens
    Log.i("VULN_APP", "Server Response: " + responseData);
}
```

Successful Response (Secure Application):

The code review shows no sensitive data being logged. Furthermore, a ProGuard rule is in place to remove all debug-level logs from the production build.

```
// From /com/example/secure/LoginActivity.java

public void onLoginClick(String username) {
    // Only non-sensitive information is logged
    Log.d("SECURE_APP", "Attempting login for user: " + username);
    // ...
}

# From proguard-rules.pro

# [INFO] This rule strips all Debug and Verbose logs from the build.
-assumenosideeffects class android.util.Log {
    public static int v(...);
    public static int d(...);
}
```

Activity 2.4: Test for Weak or Insecure Cryptography

Main Heading: MASVS-CRYPTO-1 - The app uses modern, proven cryptographic algorithms.

Sub-heading: MASTG-CRYPTO-01 - Test for Weak Crypto Algorithms

Objective: To identify the use of weak, broken, or insecurely configured cryptographic algorithms in the code.

Impact:

- **Critical Risk:** This gives a false sense of security. Data encrypted with weak crypto (like AES in ECB mode, or algorithms like DES) can be decrypted by an attacker. Weak password hashing (MD5, SHA1) allows for rapid cracking of stolen password hashes.

Tooling:

- jadx-gui
- MobSF

Methodology:

1. **Automated Scan:** MobSF's static analyzer is excellent at flagging common "Weak Crypto" issues.
2. **Manual Code Review:** In the decompiled source, search for keywords related to cryptography:
 - Cipher.getInstance
 - "AES/ECB": **(Critical)** The ECB mode is insecure as it lacks diffusion and leaks data patterns.
 - "DES" or "3DES": **(High)** These are deprecated, weak algorithms.
 - MessageDigest.getInstance: Check for "MD5" or "SHA-1". These are insecure for password hashing.
 - IvParameterSpec: Check if a **static or hardcoded IV** is being used. An Initialization Vector (IV) must be random and unique.
 - SecureRandom: Check if a weak or predictable Random Number Generator (RNG) is used instead of a cryptographically secure one.

Unsuccessful Response (Vulnerable Application):

The code review shows the app is using ECB mode and a static, hardcoded IV, making its encryption trivial to break.

```
// From /com/example/vulnerable/CryptoManager.java

public class CryptoManager {
    // [!!!] CRITICAL: Hardcoded IV is a major cryptographic flaw
    private static final byte[] STATIC_IV =
        new byte[]{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15 };

    public String encrypt(String data) {
```

```
// [!!!!] CRITICAL: Use of AES/ECB mode is insecure
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");

IvParameterSpec ivSpec = new IvParameterSpec(STATIC_IV);
cipher.init(Cipher.ENCRYPT_MODE, this.secretKey, ivSpec);
// ...
}
}
```

Successful Response (Secure Application):

The application uses modern, recommended algorithms and modes, such as AES-GCM, and correctly generates a random IV for each encryption.

```
// From /com/example/secure/CryptoManager.java

public class CryptoManager {
    // [INFO] Using GCM is a modern, authenticated encryption mode
    private static final String ALGORITHM = "AES/GCM/NoPadding";

    public String encrypt(String data) {
        Cipher cipher = Cipher.getInstance(ALGORITHM);

        // [INFO] A new, random IV is generated for every encryption
        byte[] iv = new byte[cipher.getBlockSize()];
        SecureRandom random = new SecureRandom();
        random.nextBytes(iv);

        GCMParameterSpec gcmSpec = new GCMParameterSpec(128, iv);
        cipher.init(Cipher.ENCRYPT_MODE, this.secretKey, gcmSpec);
        // ...
    }
}
```

This completes the static analysis phase. We have now found vulnerabilities by just reading the code. The next step, **Phase 3: Dynamic & Network Analysis**, will involve running the app on a test device to intercept its traffic and analyze its runtime behavior.

Phase 3: Dynamic & Network Analysis (Runtime)

Objective: To analyze the application while it is running on a device or emulator. This phase is critical for intercepting data-in-transit, assessing runtime data leakage, and interacting with the app's components to find vulnerabilities that are not visible in the source code. This is the foundation for all API testing.

Activity 3.1: Intercept All Network Traffic (HTTP & HTTPS)

Main Heading: MASVS-NETWORK-1 - The app secures all network traffic.

Sub-heading: MASTG-TEST-0236 - Cleartext Traffic Observed on the Network.

Objective: To set up a Man-in-the-Middle (MITM) proxy to intercept, view, and modify all network requests made by the application. This is the single most important step for API analysis.

Impact:

- **Critical:** If an app sends credentials or session tokens over unencrypted **HTTP**, they can be instantly stolen. Intercepting **HTTPS** is the necessary first step to test for all other API vulnerabilities (IDOR, SQLi, etc.).

Tooling:

- Burp Suite
- Android Emulator (with root) or a Rooted Physical Device
- adb (Android Debug Bridge)

Methodology:

1. Configure Burp Suite:

- Go to the "Proxy" -> "Options" tab.
- Select your workstation's IP address and add a new listener (e.g., on port 8080). Ensure "Listen on all interfaces" is enabled.

2. Configure Android Device:

- On your Android device, go to Wi-Fi settings.
- Modify your current network, set "Proxy" to "Manual".
- Enter your workstation's IP as the "Proxy hostname" and 8080 as the "Proxy port".

3. Install Burp CA Certificate:

- On the Android device, open a browser and navigate to http://burp (or your proxy IP).
- Download the "CA Certificate" (cacert.der).
- Go to "Settings" -> "Security" -> "Encryption & credentials" -> "Install a certificate" and install the cacert.der file. (Note: On modern Android, this requires moving the cert to the system trust store, which requires root).

4. Observe Traffic:

Open the target application and use its features. You should see all network traffic appearing in Burp Suite's "Proxy" -> "HTTP history" tab.

Unsuccessful Response (Vulnerable - HTTP):

Burp Suite's history shows requests being sent over **HTTP**. The "Raw" request tab shows sensitive data (like a password) in plaintext.

```
POST /login HTTP/1.1
Host: http-api.example.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 35

username=admin&password=P@ssword123!
```

Successful Response (Secure - HTTPS):

The application refuses to send cleartext traffic. All requests in Burp Suite are to **HTTPS** endpoints. (Note: You will likely see SSL/TLS errors at this stage if the app uses SSL Pinning. This is the "Successful Response" for this test, and we will bypass it in the next activity.)

Activity 3.2: Bypass SSL/TLS Pinning

Main Heading: MASVS-NETWORK-2 - The app performs identity pinning.

Sub-heading: MASTG-TEST-0068 - Testing Certificate Pinning.

Objective: To bypass the application's client-side certificate pinning. Pinning is a security control that prevents proxying, so we must defeat it to continue our testing.

Impact:

- This is a **test of a security control**. A "Win" for the tester means bypassing the control and gaining full visibility into the app's API. A "Loss" (failure to bypass) means the app is well-defended, but our testing is severely limited.

Tooling:

- `frida` (and `frida-server` installed on the rooted device)
- `objection` (a runtime patching toolkit built on Frida)
- `Burp Suite` (running as a proxy)

Methodology:

1. **Confirm Pinning:** With the Burp proxy active (from Activity 3.1), run the app. If the app fails to load, shows a network error, and Burp's "Alerts" tab shows "The client failed to negotiate a TLS connection," the app is likely using SSL Pinning.
2. **Install `frida-server`:** Download the correct `frida-server` binary for your device's architecture (e.g., `arm64`), push it to the device, and run it as root:

```
adb push frida-server /data/local/tmp/
adb shell "su -c /data/local/tmp/frida-server &"
```

3. **Install `objection`:** On your workstation: `pip install objection`

4. **Run the Bypass:** Find the app's package name (e.g., `com.example.vulnerable`) and use `objection` to attach and run its built-in bypass script:

```
objection -g com.example.vulnerable explore
```

5. Inside the `objection` shell, run the bypass command:

```
(com.example.vulnerable) > android sslpinning disable
```

6. `objection` will automatically hook common pinning methods. Relaunch the app and observe Burp Suite.

Unsuccessful Response (Tester's "Win"):

The `objection` command succeeds, and Burp Suite's "HTTP history" tab now populates with fully decrypted HTTPS requests and responses from the application's API.

```
(com.example.vulnerable) > android sslpinning disable
(agent) Hooking modules for SSL pinning detection...
(agent) Pinning disabled for 'okhttp3.CertificatePinner'
(agent) Pinning disabled for 'TrustManagerImpl'
(agent) ...
```

Successful Response (Secure Application / Tester's "Loss"):

The `objection` script fails. The app may detect Frida (an anti-reversing defense) and crash, or it may use an obscure or native-code pinning method that `objection`'s generic script cannot find.

Activity 3.3: Test for Sensitive Data in `logcat` (Dynamic)

Main Heading: `MASVS-STORAGE-2` - The app prevents leakage of sensitive data.

Sub-heading: `MASTG-TEST-0203` - Runtime Use of Logging APIs.

Objective: To dynamically confirm the static analysis finding (Activity 2.3) by observing the live system log (`logcat`) for sensitive data while using the app.

Impact:

- **High Risk:** An attacker with `adb` access (or a malicious app with `READ_LOGS` permission on older Android versions) can monitor `logcat` and steal credentials, tokens, or PII as they are being used.

Tooling:

- `adb`

Methodology:

1. Connect to your test device with `adb`.

2. Open your terminal and start `logcat`. It is best to clear old logs and filter by the app's Process ID (PID) or package name to reduce noise.

```
# Get the PID
PID=$(adb shell pidof com.example.vulnerable)

# Start logcat filtering by that PID
adb logcat --pid=$PID
```

3. Use the application, focusing on all sensitive functions:
 - Log in and log out.
 - Change your password.
 - View your user profile or personal data.
 - Make a transaction.
4. Carefully observe the terminal output for any sensitive data.

Unsuccessful Response (Vulnerable Application):

The live `logcat` output clearly shows sensitive data in plaintext as you use the app.

```
D/VULN_APP: Attempting login for user: admin
D/VULN_APP: User's password is: P@ssword123! # [!!!!] CRITICAL
I/VULN_APP: Server Response: {"token":"eyJh...[etc]"} # [!!!!] HIGH
I/VULN_APP: User loaded profile: { "name":"test", "ssn":"123-45-678" }
```

Successful Response (Secure Application):

The `logcat` output shows only generic or non-sensitive messages. Sensitive data is never logged.

```
D/VULN_APP: LoginActivity: onCreate
D/VULN_APP: Login button clicked
I/VULN_APP: Login request sent.
I/VULN_APP: Profile data loaded.
```

Activity 3.4: Test for Sensitive Data Exposure in App Switcher

Main Heading: `MASVS-STORAGE-2` - The app prevents leakage of sensitive data.

Sub-heading: `MASTG-TEST-0289` - Runtime Verification of Sensitive Content Exposure in Screenshots During App Backgrounding.

Objective: To check if the app's screen content (which may contain sensitive data) is visible in the device's "recent apps" switcher (overview screen).

Impact:

- **Medium Risk:** Any person with physical access to the device can open the app switcher and view sensitive information (like balances, PII, or messages) without being authenticated.

Tooling:

- Android device/emulator.

Methodology:

1. Navigate to a sensitive screen in the application (e.g., "My Profile," "Account Balance," "Change Password").
2. Press the "Recents" or "Overview" button on the device (the square button).
3. Observe the screenshot (snapshot) of the app in the app switcher.

Unsuccessful Response (Vulnerable Application):

The app's sensitive data is clearly visible in the screenshot in the app switcher. An attacker could glance at the overview screen and steal a credit card number, balance, or PII.

Successful Response (Secure Application):

The app's screenshot is blank, grayed out, or covered by a splash screen/logo. This is achieved by the developer setting the `FLAG_SECURE` layout parameter for that Activity.

Activity 3.5: Test for Insecure Component Interaction (Exported Activities)

Main Heading: `MASVS-PLATFORM` - Platform Interaction

Sub-heading: `MASTG-PLAT-04` - Testing Exported Activities

Objective: To dynamically launch exported activities (which we identified in Activity 1.3) to see if they can be used to bypass security controls, such as the login screen.

Impact:

- **Critical Risk:** If an app's "main dashboard" or "user profile" activity is exported, an attacker can write a malicious app that launches it directly, bypassing the login activity entirely and gaining unauthorized access to the victim's session.

Tooling:

- `adb` (Activity Manager `am`)
- `drozer`

Methodology:

1. Use the list of exported activities found in Activity 1.3 (e.g., `com.example.vulnerable.DashboardActivity`).
2. Ensure the app is not running or you are logged out.

3. Use the `adb` shell `am` (activity manager) command to try and launch the activity directly by specifying its full component name:

```
# -n {package_name}/{full_activity_name}
adb shell am start -n
com.example.vulnerable/com.example.vulnerable.DashboardActivity
```

4. Alternatively, use `drozer`:

```
dz> run app.activity.start --component com.example.vulnerable
com.example.vulnerable.DashboardActivity
```

5. Observe the device. Does the `DashboardActivity` launch and show you sensitive data?

Unsuccessful Response (Vulnerable Application):

The `DashboardActivity` launches, and you are presented with the authenticated app interface, having completely bypassed the `LoginActivity`. The app assumed that only the `LoginActivity` could ever launch the dashboard.

Successful Response (Secure Application):

The app launches but immediately detects there is no valid session, and redirects you back to the `LoginActivity`. A secure app performs a session check in the `onCreate()` or `onResume()` method of every sensitive activity.

(Note: The most secure response is for the activity not to be exported at all if it's not intended to be launched by other apps.)

This completes the dynamic and network reconnaissance phase. We have now established a full MITM proxy, bypassed SSL pinning, and confirmed several runtime data leaks. We are now ready to move to **Phase 4: API & Network Attack**, where we will use our intercepting proxy to actively tamper with API requests.

Phase 4: API & Network Attack (Server-Side)

Objective: To use our established Man-in-the-Middle (MITM) position (from Phase 3) to actively manipulate API requests. This phase tests the server's logic and its resilience to tampered input, focusing on critical flaws like broken access control, injection vulnerabilities, and other API-specific weaknesses.

Activity 4.1: Test for Broken Object Level Authorization (BOLA / IDOR)

Main Heading: `MASVS-AUTH-1` - Authentication and authorization are performed server-side.

Sub-heading: `MASTG-AUTH-04` - Testing for Insecure Direct Object References

Objective: To determine if the application's API allows a user to access resources (like profiles, messages, or files) belonging to *another* user simply by changing an identifier in the API request.

Impact:

- **Critical Risk:** 🌟 This is one of the most common and damaging API vulnerabilities. A successful BOLA attack can lead to a complete compromise of all user data, allowing an attacker to read, modify, or delete any information in the system.

Tooling:

- `Burp Suite` (specifically, the "Repeater" tool)
- Two different user accounts (User A and User B)

Methodology:

1. Log in as **User A** and ensure your traffic is proxied through Burp Suite.
2. Use the app to perform an action that fetches User A's private data (e.g., view profile, load chat history).
3. In Burp Suite's "HTTP history," find this request (e.g., `GET /api/v1/users/101/profile`). Note the ID `101` belongs to User A.
4. Right-click this request and "Send to Repeater."
5. In Repeater, modify the request. Change the user ID from User A's (`101`) to User B's (`102`).
6. **Crucially, do not change the `Authorization` (token/cookie) header.** You are attacking User B's endpoint *while authenticated as User A*.
7. Send the modified request and analyze the response.

Unsuccessful Response (Vulnerable Application):

The server returns a `200 OK` status and the response body contains User B's private, sensitive information. This confirms the server only checked for a valid session, not for ownership of the resource.

```
-- ATTACKER'S REQUEST --
GET /api/v1/users/102/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer [User_A_Session-Token]

-- VULNERABLE RESPONSE --
HTTP/1.1 200 OK
Content-Type: application/json

{
  "userId": 102,
  "username": "UserB",
  "email": "userB_private@example.com",
  "ssn": "123-45-678"
}
```

Successful Response (Secure Application):

The server correctly denies the request because User A is not authorized to view resource `102`.

```
-- SECURE RESPONSE --
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "error": "You are not authorized to access this resource."
}
```

Activity 4.2: Test for Broken Function Level Authorization (BFLA)

Main Heading: `MASVS-AUTH-1` - Authentication and authorization are performed server-side.

Sub-heading: `MASTG-AUTH-03` - Testing for Privilege Escalation

Objective: To verify that a low-privilege user cannot access API endpoints that should be reserved for high-privilege users (e.g., administrators).

Impact:

- **Critical Risk:** 🚫 A successful BFLA attack allows a standard user to perform administrative actions, such as deleting other users, changing application settings, or viewing site-wide analytics, leading to a full compromise.

Tooling:

- `Burp Suite` (Repeater)
- A standard user account (User A)
- (Ideally) An admin user account (to discover admin endpoints)

Methodology:

1. First, map the application using your admin account (if you have one) to discover the paths for admin-only functions (e.g., `POST /api/admin/addUser`, `DELETE /api/v1/users/102`).
2. If you do *not* have an admin account, you must guess common administrative paths (e.g., `/admin`, `/api/v1/admin`, `/api/v1/users`).
3. Log in as the **standard user (User A)** and capture their session token.
4. Go to Burp Repeater.
5. Craft a request to one of the admin-only endpoints you discovered (e.g., `DELETE /api/v1/users/102`), but use the `Authorization` token belonging to **User A**.
6. Send the request and analyze the response.

Unsuccessful Response (Vulnerable Application):

The server returns a `200 OK` or `204 No Content` response, and the action is *successful* (e.g., User B is deleted). This proves the endpoint checked for a valid token but failed to check the user's *role*.

```
-- ATTACKER'S REQUEST (AS STANDARD USER) --
DELETE /api/v1/users/102 HTTP/1.1
Host: api.example.com
Authorization: Bearer [User_A_Session-Token]

-- VULNERABLE RESPONSE --
HTTP/1.1 204 No Content
```

Successful Response (Secure Application):

The server correctly checks the user's role, denies the request, and returns an authorization error.

```
-- SECURE RESPONSE --
HTTP/1.1 403 Forbidden
Content-Type: application/json

{
  "error": "Administrative privileges required to perform this action."
}
```

Activity 4.3: Test for SQL Injection (SQLi)

Main Heading: `MASVS-CODE-3` - The app does not rely on client-side input validation.

Sub-heading: `MASTG-TEST-0025` - Testing for Injection Flaws.

Objective: To determine if an attacker can manipulate API parameters to inject and execute arbitrary SQL commands on the backend database.

Impact:

- **Critical Risk:** SQL Injection is a game-over vulnerability. It can lead to a complete compromise of the database, allowing an attacker to steal all user data, modify data, or even gain Remote Code Execution (RCE) on the server.

Tooling:

- `Burp Suite` (Repeater and Intruder)
- `sqlmap` (for automated exploitation)

Methodology:

1. In Burp Suite, identify an API endpoint that takes user input to query data. A search function is a perfect target.

```
GET /api/v1/products/search?query=test
```

2. Send this request to Burp Repeater.

3. **Method 1: Error-Based.** Inject a single quote (') into the `query` parameter:

```
GET /api/v1/products/search?query=test'
```

4. Analyze the response. Look for a `500 Internal Server Error` or a verbose database error message in the JSON body.

5. **Method 2: Time-Based (Blind).** If the app handles errors gracefully, try to force a time-delay.

- **Payload (PostgreSQL):** `...search?query=test' AND 1=(SELECT 1 FROM PG_SLEEP(5))--`

- **Payload (MySQL):** `...search?query=test' AND (SELECT 1 FROM (SELECT(SLEEP(5)))a)--`

6. Send the request. If the server response is delayed by 5 seconds, it is vulnerable.

Unsuccessful Response (Vulnerable Application):

The server returns a detailed database error, confirming the injection.

```
-- ATTACKER'S REQUEST --
GET /api/v1/products/search?query=test' HTTP/1.1
Host: api.example.com
...

-- VULNERABLE RESPONSE --
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{
  "error": "SQL Exception: You have an error in your SQL syntax;
           check the manual that corresponds to your MySQL server
           version for the right syntax to use near ''' at line 1"
}
```

(Alternatively, a 5-second delay in the response would also be an "Unsuccessful Response".)

Successful Response (Secure Application):

The server uses parameterized queries (prepared statements). It treats the single quote as a literal character, finds no matching results, and returns an empty list.

```
-- SECURE RESPONSE --
HTTP/1.1 200 OK
Content-Type: application/json

{
  "results": []
}
```

Activity 4.4: Test for Stored XSS in a `WebView`

Main Heading: `MASVS-PLATFORM-2` - The app securely interacts with `WebView`s.

Sub-heading: `MASTG-PLAT-08` - Testing for `WebView` Cross-Site Scripting

Objective: To inject a malicious script (XSS payload) into an API field (e.g., user profile bio, comment) and confirm it executes when that data is rendered inside a `WebView` in the mobile app.

Impact:

- **High Risk:** A successful XSS in a `WebView` can be *more* dangerous than in a web browser. The script executes within the app's context and can be used to steal session tokens or abuse exposed Java "JavaScript Bridges" (see Phase 5) to access native phone features or local files.

Tooling:

- `Burp Suite` (Repeater)
- Android Device (to observe the XSS)

Methodology:

1. Identify a feature where you can submit data that is displayed back to you (e.g., "Edit Profile," "Post Comment").
2. In Burp Repeater, intercept the request that saves this data (e.g., `PUT /api/v1/profile`).
3. In the JSON body, modify a field to include a basic XSS payload.

```
{
  "username": "UserA",
  "bio": "Just a test user <img src=x onerror=alert('XSS-from-API')>"
}
```

4. Send the request. The server will store this malicious string.
5. On your device, navigate to the screen that displays this data (e.g., your "View Profile" screen, which is a `WebView`).
6. Observe the device for a pop-up alert box.

Unsuccessful Response (Vulnerable Application):

When you navigate to the profile screen, a native Android alert box pops up with the text "XSS-from-API". This confirms the `WebView` rendered the HTML, and the script executed.

Successful Response (Secure Application):

The application is secure in one of two ways:

1. **Server-Side:** The server sanitized the input, and the API response shows the payload was stripped or encoded.

2. **Client-Side:** The app displays the literal, harmless text in the `WebView`: `Just a test user <img src=x...>` (e.g., it used `TextView` instead of a `WebView`, or the `WebView` has JavaScript disabled).
-

Activity 4.5: Test for Mass Assignment

Main Heading: `MASVS-AUTH-1` - Authentication and authorization are performed server-side.

Sub-heading: `MASTG-AUTH-08` - Testing for Mass Assignment

Objective: To exploit API endpoints that update objects by "guessing" and adding hidden, privileged properties to the JSON request body, such as `isAdmin` or `balance`.

Impact:

- **Critical Risk:** A successful mass assignment attack allows a standard user to promote themselves to an administrator, change their account balance, or bypass other server-side logic simply by sending extra parameters in a request.

Tooling:

- `Burp Suite` (Repeater)

Methodology:

1. Identify an API endpoint that creates or updates a data object, such as user registration or profile update.

```
PUT /api/v1/profile/update
```

2. Capture a legitimate request in Burp Repeater. The body will have known, safe fields:

```
{
  "username": "test_user",
  "first_name": "Test",
  "last_name": "User"
}
```

3. Modify the JSON body to "guess" and inject privileged fields.

```
{
  "username": "test_user",
  "first_name": "Test",
  "last_name": "User",
  "isAdmin": true,
  "role": "admin",
  "account_balance": 999999
}
```

4. Send the modified request.

5. Analyze the response. Even if it's a `200 OK`, you must verify. Make a *separate* `GET /api/v1/profile` request to see if the privileged data was saved.

Unsuccessful Response (Vulnerable Application):

You send the modified request. You then fetch your profile, and the server's response confirms you are now an admin.

```
-- VERIFICATION REQUEST --
GET /api/v1/profile HTTP/1.1
Host: api.example.com
Authorization: Bearer [User_A_Session-Token]

-- VULNERABLE RESPONSE --
HTTP/1.1 200 OK
Content-Type: application/json

{
  "username": "test_user",
  "first_name": "Test",
  ...
  "role": "admin",
  "account_balance": 999999
}
```

Successful Response (Secure Application):

The server's API code uses a "whitelist" model (or DTOs - Data Transfer Objects) and only maps the expected fields (`username`, `first_name`, `last_name`). The extra fields are ignored. Your verification request shows no change in privilege.

```
-- SECURE RESPONSE --
HTTP/1.1 200 OK
Content-Type: application/json

{
  "username": "test_user",
  "first_name": "Test",
  ...
  "role": "user",
  "account_balance": 10.00
}
```

This completes the server-side API attack phase. We have now attacked the API endpoints from an authenticated, external perspective. The next phase, **Phase 5: Advanced Local & Runtime Analysis**,

will involve "hooking" the running application's memory to bypass client-side controls and further exploit the app.

Phase 5: Advanced Local & Runtime Analysis (Client-Side)

Objective: To interact directly with the application in a rooted, dynamic environment. This phase involves using runtime instrumentation tools like **Frida** and **Objection** to hook into the app's memory, bypass client-side controls (like root detection and biometric auth), and exploit vulnerabilities that are only present at runtime.

Activity 5.1: Access & Steal Sandboxed Data (Dynamic)

Main Heading: `MASVS-STORAGE-1` - The app securely stores sensitive data.

Sub-heading: `MASTG-TEST-0207` - Data Stored in the App Sandbox at Runtime.

Objective: To dynamically confirm the static findings (from Activity 2.1) by accessing the app's private sandbox directory on a rooted device to read and steal sensitive files.

Impact:

- **Critical Risk:** If an attacker gains root access (or has physical access to the device and can achieve it), they can steal plaintext credentials, tokens, and databases stored insecurely in the app sandbox. This completely bypasses all app logic.

Tooling:

- `adb` (Android Debug Bridge)
- A rooted device or emulator

Methodology:

1. Connect your rooted device and gain a root shell:

```
adb shell
su
```

2. Navigate to the target application's private data directory:

```
cd /data/data/com.example.vulnerable
```

3. List the contents, looking for `shared_prefs`, `databases`, and `files` directories:

```
ls -laR
```

4. Examine the `SharedPreferences` XML file we found in Activity 2.1:

```
cat shared_prefs/Session.xml
```

5. If you find a database, dump its contents (e.g., a `user.db` file):

```
sqlite3 databases/user.db "SELECT * FROM users;"
```

Unsuccessful Response (Vulnerable Application):

The shell output shows the plaintext credentials we suspected from the static analysis, proving a critical data storage flaw.

```
# cat /data/data/com.example.vulnerable/shared_prefs/Session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="username">admin</string>
  <string name="password">P@ssword123!</string> <string
name="session_token">eyJh...[etc]</string> </map>
```

Successful Response (Secure Application):

The data is properly encrypted. Using `cat` on the `SharedPreferences` file shows undecipherable, Base64-encoded encrypted data (if using `EncryptedSharedPreferences`).

```
# cat /data/data/com.example.secure/shared_prefs/secure_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="session_token">AQE...[long_encrypted_string]</string>
</map>
```

Activity 5.2: Bypass Root Detection

Main Heading: `MASVS-RESILIENCE-1` - The app implements defenses against tampering.

Sub-heading: `MASTG-RESILIENCE-01` - Testing for Root Detection

Objective: To bypass the application's client-side check that prevents it from running on a rooted device. This is a required step to enable the use of advanced tools like Frida.

Impact:

- **High Risk (Control Bypass):** Bypassing root detection is a prerequisite for most advanced client-side attacks, as tools for instrumentation, hooking, and dynamic analysis require root access.

Tooling:

- `frida` (and `frida-server`)
- `objection`
- A rooted device or emulator

Methodology:

1. Launch the app on your rooted device. Observe that it likely shows an error message (e.g., "Rooted device detected. App will now exit.") and closes.
2. Ensure `frida-server` is running on your rooted device (from Activity 3.2).
3. On your workstation, attach `objection` to the app's process. Use the `-g` (gadget) flag to spawn the app and attach immediately:

```
objection -g com.example.vulnerable explore
```

4. Inside the `objection` shell, run the generic root bypass command. This hooks common root-detection methods and forces them to return `false`.

```
(com.example.vulnerable) > android root disable
```

5. The app should now load successfully, having bypassed the check.

Unsuccessful Response (Vulnerable Application):

`objection` successfully hooks the methods. The app's root-check dialog disappears, and the application becomes fully usable on the rooted device.

```
(com.example.vulnerable) > android root disable
(agent) Hooking modules for root detection...
(agent) Bypassing Common root detection method (File.exists)
(agent) Bypassing Common root detection method (PackageManager)
(agent) ...
Job: 1 (com.example.vulnerable) | Bypasses enabled
```

Successful Response (Secure Application):

The `objection` script fails. The app may use advanced or native-code (JNI) checks that are not easily hooked, or it may detect Frida itself (Frida detection) and crash. This forces the tester to write a more complex, custom Frida script.

Activity 5.3: Exploit Insecure `WebView` JavaScript Bridge

Main Heading: `MASVS-PLATFORM-2` - The app securely interacts with `WebView`s.

Sub-heading: `MASTG-TEST-0033` - Testing for Java Objects Exposed Through WebViews.

Objective: To exploit a `WebView` that has a "JavaScript Bridge" (`addJavascriptInterface`) enabled, allowing malicious JavaScript (e.g., from a Stored XSS) to call native Java code.

Impact:

- **Critical Risk:** 🌟 A vulnerable JavaScript bridge allows a simple XSS payload to "break out" of the `WebView` sandbox. This can lead to Remote Code Execution (RCE) by allowing the script to read/write local files, access the app's private database, or steal session tokens directly from the native Java code.

Tooling:

- `jadx-gui` (to find the bridge)
- `Burp Suite` (to inject the XSS payload)
- Android Device (to observe the exploit)

Methodology:

1. **Static Analysis:** First, review the decompiled code (from Phase 2). Search for the string `addJavascriptInterface`.
2. Analyze the Java class being exposed. Look for any dangerous, exposed methods.

```
// In /com/example/vulnerable/MyWebViewActivity.java
WebView myWebView = (WebView) findViewById(R.id.webview);
myWebView.getSettings().setJavaScriptEnabled(true);

// [!!!] CRITICAL: Exposing a dangerous object under the name "Android"
myWebView.addJavascriptInterface(new WebAppInterface(this), "Android");

// The exposed class:
public class WebAppInterface {
    // [!!!] CRITICAL: This method can be called from JavaScript
    @JavascriptInterface
    public String getSessionToken() {
        return SessionManager.getPersistentToken();
    }
}
```

3. **Dynamic Exploitation:** Use the Stored XSS vulnerability you found in Activity 4.4. Instead of a simple `alert()`, inject a payload that calls this exposed bridge:

```
... "bio": "<img src=x onerror=alert(Android.getSessionToken())>" ...
```

4. Submit the request via Burp, then navigate to the profile page in the app.

Unsuccessful Response (Vulnerable Application):

An alert box pops up on the device, but this time it contains the user's **actual session token**, which was stolen from the app's native Java code.

(The alert box displays: `eyJh...[Long_session_token]...`)

This confirms the XSS payload successfully called the native Java bridge and exfiltrated critical credentials.

Successful Response (Secure Application):

The application either does not use `addJavascriptInterface` or, if it does, it correctly restricts which methods are exposed using the `@JavascriptInterface` annotation (on API levels < 17) and only

exposes non-sensitive helper functions. The XSS payload fails to find the `Android` object or the `getSessionToken` method.

Activity 5.4: Bypass Biometric Authentication

Main Heading: `MASVS-AUTH-3` - Biometric authentication, if used, is not bypassable.

Sub-heading: `MASTG-AUTH-10` - Bypassing Biometric Authentication

Objective: To bypass the client-side biometric (fingerprint/face) check using runtime instrumentation.

Impact:

- **High Risk:** If an attacker gains physical access to a logged-in but locked device, they are often stopped by a biometric prompt before accessing a critical function (e.g., "Confirm Payment"). Bypassing this check allows them to complete the malicious action.

Tooling:

- `frida`
- `objection`

Methodology:

1. Navigate to the part of the app that requires a biometric check (e.g., a "Transfer Funds" button). The Android biometric prompt appears.
2. Ensure `frida-server` is running and attach `objection` to the app:

```
objection -g com.example.vulnerable explore
```

3. Use `objection`'s built-in bypass. This command hooks the `onAuthenticationSucceeded` callback and forces it to trigger, tricking the app into thinking the user successfully scanned their fingerprint.

```
(com.example.vulnerable) > android biometrics bypass
```

4. Observe the application.

Unsuccessful Response (Vulnerable Application):

The biometric prompt immediately disappears, and the app proceeds to the next step (e.g., "Transfer Successful!"). The bypass worked.

```
(com.example.vulnerable) > android biometrics bypass
(agent) Hooking BiometricPrompt.AuthenticationCallback.onAuthenticationSucceeded...
(agent) Forcing biometrics bypass...
(agent) Bypass enabled for 1 hook(s)
```

Successful Response (Secure Application):

The app is secure if:

1. The `objection` script fails to find the callback (less likely).
2. **True Security (Server-Side Check):** The app does not rely *only* on the client-side biometric check. After the client-side "success" (which we faked), the app *still* requires a server-side re-authentication (e.g., the password) or a separate, valid one-time token before performing the critical action. Our faked bypass is insufficient to complete the transaction.

This completes the advanced runtime analysis phase. We have now compromised client-side storage, bypassed runtime defenses, and exploited weaknesses in the app's local logic. The final phase, **Phase 6: Resilience & Obfuscation Analysis**, will focus on the app's defenses against our testing methods.

Phase 6: Resilience & Anti-Reversing Analysis

Objective: To test the app's defenses against the very techniques we are using. A resilient app will actively try to detect and block reverse engineering (static analysis) and runtime instrumentation (dynamic analysis). Our goal is to identify these defenses and attempt to bypass them.

Activity 6.1: Test for Code Obfuscation

Main Heading: `MASVS-RESILIENCE-2` - The app implements defenses against reverse engineering.

Sub-heading: `MASTG-RESILIENCE-03` - Testing for Obfuscation

Objective: To verify that the application's code has been properly obfuscated to make static analysis difficult.

Impact:

- **Medium Risk (Defense):** A lack of obfuscation makes an attacker's job trivial. It allows them to easily read the decompiled code, understand the business logic, and find vulnerabilities (as we did in Phase 2). Strong obfuscation is a critical defense-in-depth control.

Tooling:

- `jadx-gui`

Methodology:

1. Open the target `.apk` file in `jadx-gui` (as performed in Activity 1.2).
2. Browse the package names and class names.
3. **Analyze Names:** Look at the names of classes, methods, and variables.
 - Are they descriptive and readable (e.g., `LoginActivity`, `checkPassword`, `ApiClient`)?
 - Or are they short, meaningless, and sequential (e.g., `a.a.b`, `a.b.c`, `x()`, `y()`)?
4. **Analyze Logic:** Look at the body of a method. Is the logic straightforward, or has it been intentionally convoluted (control flow obfuscation)?

Unsuccessful Response (Vulnerable / Unobfuscated):

The decompiled code is perfectly readable, with all original class and method names intact. This makes finding flaws simple.

```
// /com/example/vulnerable/LoginActivity.java

public class LoginActivity extends AppCompatActivity {
    private EditText usernameField;
    private EditText passwordField;

    protected void onCreate(Bundle savedInstanceState) { ... }

    public void attemptLogin() {
        String user = usernameField.getText().toString();
        String pass = passwordField.getText().toString();

        if (checkPassword(user, pass)) {
            // ...
        }
    }
}
```

Successful Response (Secure / Obfuscated):

The code is heavily obfuscated. All meaningful names are replaced with meaningless characters, forcing an attacker to trace the logic manually, which is significantly more time-consuming.

```
// /a/b/c/a.java

public class a extends androidx.appcompat.app.c {
    private EditText a;
    private EditText b;

    protected void onCreate(Bundle bundle) { ... }

    public void x() {
        String str = this.a.getText().toString();
        String str2 = this.b.getText().toString();

        if (y(str, str2)) {
            // ...
        }
    }
}
```

Activity 6.2: Test for Anti-Tampering (Signature Verification)

Main Heading: MASVS-RESILIENCE-1 - The app implements defenses against tampering.

Sub-heading: MASTG-RESILIENCE-01 - Testing for Tampering

Objective: To determine if the app validates its own digital signature at runtime. This defense prevents an attacker from modifying the app, re-packaging it, and distributing a trojanized version.

Impact:

- **High Risk (Defense):** An app without anti-tampering can be easily modified. An attacker could decompile it, inject malicious code (e.g., a keylogger, a backdoor), re-sign it with their own key, and trick users into installing it.

Tooling:

- `apktool` (to decompile/recompile)
- `keytool` (to generate a new signing key)
- `zipalign` (to align the APK)
- `apksigner` (to sign the new APK)

Methodology:

1. **Decompile:** Use `apktool` to decompile the app into its Smali and resource files.

```
apktool d /path/to/target-app.apk -o ./decoded-app
```

2. **Tamper:** Make a harmless modification. The easiest way is to edit a string in

```
res/values/strings.xml.
```

- Change `<string name="app_name">Target App</string>` to `<string name="app_name">Tampered App</string>`.

3. **Rebuild:** Use `apktool` to rebuild the application.

```
apktool b ./decoded-app -o ./tampered.apk
```

4. **Sign:** Sign the tampered APK with your own debug key (the original key is private, so we *must* use a new one).

```
# (First-time only) Generate a key
```

```
keytool -genkey -v -keystore debug.keystore -alias debug -keyalg RSA -keysize 2048 -validity 10000
```

```
# Align and sign
```

```
zipalign -v 4 ./tampered.apk ./tampered-aligned.apk
```

```
apksigner sign --ks debug.keystore ./tampered-aligned.apk
```

5. **Install & Run:** Install the `tampered-aligned.apk` on a device and launch it.

Unsuccessful Response (Vulnerable Application):

The tampered app—now signed with a different key—runs perfectly. This proves the app does not check its own signature, and an attacker can freely modify it.

Successful Response (Secure Application):

The application launches, detects the signature mismatch, and immediately exits or shows an error (e.g., "Application integrity check failed").

(Note: This defense can also be bypassed with Frida, but its presence is a strong deterrent.)

Activity 6.3: Test for Debugger Detection

Main Heading: `MASVS-RESILIENCE-2` - The app implements defenses against reverse engineering.

Sub-heading: `MASTG-RESILIENCE-02` - Testing for Debugger Detection

Objective: To verify that the app can detect when a debugger is attached to its process and respond by exiting.

Impact:

- **Medium Risk (Defense):** Attaching a debugger (`jdb` for Java, `gdb`/`lldb` for native code) is a powerful technique for analyzing runtime logic and memory. Detecting and blocking debuggers (as distinct from being `debuggable=false`) forces an attacker to use more advanced methods.

Tooling:

- `adb`
- `jdb` (Java Debugger)
- `frida-trace` (to see if the app is checking)

Methodology:

1. Run the application on your device.
2. Find the application's Process ID (PID).

```
adb shell ps -A | grep com.example.vulnerable
# Output: u0_a123    12345    ... com.example.vulnerable
```

3. Set up the Java Debug Wire Protocol (JDWP) port forward for that PID:

```
adb forward tcp:6666 jdpw:12345
```

4. Attempt to attach the `jdb` debugger from your workstation:

```
jdb -connect com.sun.jdi.SocketAttach:hostname=localhost,port=6666
```

5. **Observe the App:** Watch the application on the device.

Unsuccessful Response (Vulnerable Application):

The `jdb` debugger attaches successfully, and the application continues to run normally, allowing the attacker to begin debugging.

```
$ jdb -connect com.sun.jdi.SocketAttach:hostname=localhost,port=6666
Set uncaught java.lang.Throwable
Set deferred uncaught java.lang.Throwable
Initializing jdb ...
>
```

Successful Response (Secure Application):

As soon as `jdb` attaches, the application detects it (e.g., by calling `android.os.Debug.isDebuggerConnected()`) and immediately exits, terminating the `jdb` session.

```
$ jdb -connect com.sun.jdi.SocketAttach:hostname=localhost,port=6666
...
>
VM Disconnected
```

Activity 6.4: Test for Emulator & Root Detection (Static Review)

Main Heading: `MASVS-RESILIENCE-2` - The app implements defenses against reverse engineering.

Sub-heading: `MASTG-RESILIENCE-01` & `MASTG-RESILIENCE-02`

Objective: To identify the *specific code* used for root and emulator detection (which we bypassed in Phase 5). This helps in understanding the *quality* of the resilience checks.

Impact:

- **Medium Risk (Defense):** Simple, known checks (like checking for the `su` binary) are easily bypassed by tools like Objection. Advanced checks (e.g., custom native code checks, checking for `frida-server`) are much more difficult to defeat and show a higher level of security maturity.

Tooling:

- `jadx-gui`

Methodology:

1. In the decompiled `jadx-gui` source, perform a global search for strings related to root and emulators.
2. **Root Detection Strings:**
 - `su`

- `supersu`
- `test-keys` (checks for dev build keys)
- `rooted`
- `RootBeer` (a common root-detection library)

3. Emulator Detection Strings:

- `sdk` or `google_sdk`
- `generic`
- `emulator`
- `qemu`
- `Genymotion`

4. Analyze the code that uses these strings. Is it a simple `File.exists("/system/bin/su")`? Or is it a more complex check inside a native library (`.so` file)?

Unsuccessful Response (Vulnerable / Weak Defense):

The code review reveals simple, well-known Java checks that are trivial to bypass.

```
// From /com/example/vulnerable/RootDetector.java
public class RootDetector {
    public static boolean isDeviceRooted() {
        // [!!!] WEAK: This is a basic check
        File file = new File("/system/bin/su");
        if (file.exists()) {
            return true;
        }

        // [!!!] WEAK: This is a basic emulator check
        if (Build.FINGERPRINT.contains("generic")) {
            return true;
        }
        return false;
    }
}
```

Successful Response (Secure / Strong Defense):

The code review reveals no simple string-based checks. Instead, the app calls a native C++ method, which is much harder to reverse engineer and hook.

```
// From /com.example.secure.SecurityManager.java
public class SecurityManager {
    // [INFO] Loading a native library for security checks
    static {
```

```
        System.loadLibrary("native-security");
    }

    // [INFO] This native method is a "black box" to Java decompilers.
    // It is much harder to bypass than a Java-level check.
    public static native boolean isSecure();
}
```

This completes the six-phase Android Application Security Testing Guide.